学習ノート / Harness 全体像

InBox、Git候補、Harness方針を一つにまとめた索引。URLは省略し、名称とDictを中心に記載する。

製品の3点セット

領域	役割	現在の位置
Note	書く：メモ、シナリオ、InBox、素材参照	kanban-note01
WikiLLM資料	引く：JsonCardで概念・関係・出典を辿る	sandbox / Study / ConceptDB
技術書	読む:Wikiを土台に章、コード、図を置く	Study / 技術書目次

全体フロー

InBox → A評価 → 人間採否 → Raw / Study / Library → RPAまたは実装 → Report → Memory

01 InBox

未整理情報の入口。Raw へ昇格した資料は出典として不可侵にする。

区分	入れるもの	次の処理
改善案	Note、UI、同期、運用の提案	Task / Policy / Report
相談芽生	CODEXやAgentへ相談する仮説	論点整理 → 判断記録
Taskメモ	実装・調査する作業候補	Kanban / rpaJobsへ昇格
OSS候補	Git、Workflow、UI技術、Engine候補	RPA化 → Library化または保留
素材候補	画像、ComfyUIWorkflow、Asset設計	AIAssets / NovelEngineへ接続

規律

候補投入 → AIが学習価値・JsonCard化・技術書化・陳腐化速度・出典品質を評価 → 人間が採否 →
Raw へ入れた本文はAIが改変しない。

注記: 個別InBox本文はブラウザlocalStorageまたはFirebase同期状態に依存する。本PDFには保存構造と既知の資料カテゴリを記載する。

02 Git候補とDict

未着手候補はRPA実行依頼を通じて資料化する。

分類	候補名	Dict / 学習用途
UI	stefan-karger/solid-ui	固体雛形集:SolidJS Dark UI、コピー部品、PWA View
Harness	OpenHarness	検証環境:Agent作業、検証、Loop、Memory
MultiAgent	ai-orchestrator Topic	群体制御:Orchestrator、Worker、Queue、Workflow
Game	tower_defense / Tower-Defense-Game	防衛波制御・防衛属性盤:敵、弾、配置、ウェーブ
Engine	Phaser / PixiJS / melonJS	二次元遊戯基盤:Scene、描画、入力、Asset Pipeline
AI素材	ComfyUI_VNCCS	固定人物:立ち絵、表情、衣装、ポーズ、sprite
AI素材	ComfyUI_StoryDiffusion	物語生成:連番Scene、人物一貫性、EventCG
AI素材	StoryboardU2 / DirectorsConsole	絵コンテ・構図設計:カット割り、配置、CM試演
AI素材	ComfyUI_Workflows	背景工房:ControlNet、背景、画角、構図制御
AI素材	Confetti	自動連携:NovelGameからComfyUIを呼ぶ参考
AI素材	Comfy-Org workflow_templates	公式雛形:Workflow JSON、検証、Flux、SDXL、Video
AI素材	ComfyUI-Wan2.2-workflow	映像試演:I2V、T2V、短尺CM

03 Harnessのルールと評価基準

Harnessは作業量を増やす仕組みではなく、目標に寄与する成果物だけを残す判断機構である。

層	意味	例
Goal	最終目的	月30万円相当の開発基盤
Evaluation	目的への寄与を判定	再利用可、検証済み、製品群へ寄与
Metric	途中観測値	Component 50、Snippet 100、Engine 10
Loop	仮説を実験し学ぶ反復	仮説 → 実験 → 評価 → 学習
Review	人間またはJudge AIの評価	S/A/B/C、採用、修正、破棄
Memory	次回判断へ残す記録	成果物、採否、失敗理由、再利用先

Policy一覧

Key	ルール
POL_REUSE	新規作成前に既存資産を検索する。
POL_EXTRACT	同じ構造が3回出現したら共通化候補にする。
POL_COMPONENT	画面ではなく部品単位で設計する。
POL_ENGINE	一作のゲームではなく派生できるEngineを先に作る。
POL_TEMPLATE	完成品より再利用できる雛形を優先する。
POL_JSON / POL_SOLID	可変データをJSONで分離し、表示はStateから導出する。

次の4手

1. Core Libraryの正本位置を決める。
2. Component DB、Snippet DB、Study記事を横断検索可能にする。
3. RUB_PRODUCT、RUB_REUSE、RUB_QUALITYをJudge AI用に作る。
4. Projectごとの資産棚卸しを行い、3回以上出現した構造を候補化する。

04 GameUI候補とDesignerAgent

基盤はGameface-UI、規範はinterface-design方式、参照はGodot・Phaser完成UI、検査はScreenshot比較とする。

分類	候補名	Dict / 用途
基盤	CoherentLabs/Gameface-UI	遊技部品:SolidJSでHUD、Menu、Slider、Navigationを試作
標本	GodotGUIsamples	画面標本:余白、Focus、選択状態、画面遷移
所持品	GodotDynamicInventorySystem	所持画面:Category、装備枠、Item DB、無限Scroll
索引	awesome-phaser	実例索引:Phaser完成ゲーム、UIplugin、Template
索引	awesome-godot	資産索引:Godotplugin、公開ゲーム、Add-on
規範	interface-design	意匠記憶:Spacing、Color、DepthをSession間で保持
知識	claude-code-game-development	制作指南:HUD、Menu、Dialog、Inventory、Accessibility
分業	gamestudio-subagents	分業制作:Design、Art、Programming、QA
役割	agency-agents	専門職群:GameDesigner、FrontendDeveloperの契約
研究	GameUIAgent	自動意匠:DesignSpecJSON、Figma、VLMReview

05 GameUIDesignerAgent

DesignerAgentを増やさず、既存Agent内のGAME_UI
Stateとして運用する。自然言語から直接コードを書かず、ui_spec.jsonを中間契約にする。

State	Action	Output
reference	類似ゲームと実装を最低3件収集	ui_reference.json
wireframe	情報優先度と操作導線を定義	ui_wireframe.json
styling	世界観を材質、形状、文字へ変換	ui_visual_spec.json
implementation	SolidJS Componentとして実装	ui_components/
review	Screenshotを規範と比較	ui_review.json

AI風UI抑制規則

Key	Rule
均等禁止	全Cardを同じ幅、高さ、余白にしない
紫色禁止	根拠のない青紫Gradientを使わない
角丸制限	全要素へ同じborder-radiusを適用しない
世界接続	装飾を世界の材質、文化、時代、道具から導出
主役限定	一画面の視覚的主役は一つ
密度差分	Title、HUD、Inventory、Dialogで密度を変える
実例参照	実装前に3件の採用点・不採用点を記録
画像検査	コードだけで完了せずScreenshotを評価

Workflow

自然言語 → ui_reference.json → ui_spec.json → SolidJS / Figma → Screenshot → ui_review.json → 修正